

Security Audit

Report for Burrowland, Ref - Dcl, Ref - Exchange

Date: May 9, 2026 **Version:** 1.6

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About Target Contracts	1
1.2 Disclaimer	2
1.3 Procedure of Auditing	3
1.3.1 Software Security	3
1.3.2 DeFi Security	3
1.3.3 NFT Security	4
1.3.4 Additional Recommendation	4
1.4 Security Model	4
Chapter 2 Findings	6
2.1 Software Security	7
2.1.1 Failure to remove position timestamp from <code>position_latest_actions</code>	7
2.2 DeFi Security	8
2.2.1 Lack of sender account check when handling <code>SwapReference</code> message	8
2.2.2 Lack of lock when decreasing collateral	9
2.2.3 Incorrect enumeration of tokens requiring price feeds	10
2.2.4 Unreasonable <code>pos_id</code> design	12
2.2.5 Lack of reasonable configuration check	15
2.2.6 Potential panic during handling message <code>SwapReference</code>	16
2.2.7 Unreasonable check of reserves	21
2.2.8 Potential sandwich attack in force close position token swap	25
2.2.9 Unreasonable leverage rate computation	29
2.2.10 Potential gas waste due to improper fee validation	29
2.2.11 Lack of account lock check in withdrawal and collateral operations	30
2.2.12 Potential underflow in function <code>internal_storage_balance_of()</code> after forced storage adjustment	35
2.2.13 Incorrect aggregation of intermediate swap outputs	36
2.3 Additional Recommendation	37
2.3.1 Automatically construct swap indication from the token information	37
2.3.2 Use <code>UnorderedMap</code> for <code>margin_positions</code> instead of <code>HashMap</code>	43
2.3.3 Incorrect error message in <code>get_token_out()</code>	44
2.3.4 Lack of check in function <code>update_asset()</code>	44
2.3.5 Remove redundant code in function <code>internal_decrease_collateral_for_gap()</code>	45
2.4 Notes	47
2.4.1 Potential centralization risks	47
2.4.2 Collateral tracking approximation due to legacy asset accounting	47

Report Manifest

Item	Description
Client	Ref Finance
Target	Burrowland, Ref - Dcl, Ref - Exchange

Version History

Version	Date	Description
1.0	May 22, 2024	First release
1.1	Jan 2, 2025	Second release
1.2	Apr 2, 2025	Third release
1.3	Sep 5, 2025	Fourth release
1.4	Nov 6, 2025	Fifth release
1.5	Jan 12, 2026	Sixth release
1.6	May 9, 2026	Seventh release

Signature

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Rust
Approach	Semi-automatic and manual verification

The target of this audit is the code repository of Burrowland¹ (migrated to a new repository in Version 9²), Ref-Dcl³, Ref-Exchange⁴ of Ref Finance. Note that, we did **NOT** audit all the modules in the repository. Specifically, the files covered in this audit include:

```
1 burrowland/contracts/contract/src/upgrade.rs
2 burrowland/contracts/contract/src/events.rs
3 burrowland/contracts/contract/src/price_receiver.rs
4 burrowland/contracts/contract/src/legacy.rs
5 burrowland/contracts/contract/src/config.rs
6 burrowland/contracts/contract/src/lib.rs
7 burrowland/contracts/contract/src/account_view.rs
8 burrowland/contracts/contract/src/margin_actions.rs
9 burrowland/contracts/contract/src/margin_trading.rs
10 burrowland/contracts/contract/src/margin_pyth.rs
11 burrowland/contracts/contract/src/margin_config.rs
12 burrowland/contracts/contract/src/fungible_token.rs
13 burrowland/contracts/contract/src/big_decimal.rs
14 burrowland/contracts/contract/src/margin_accounts.rs
15 burrowland/contracts/contract/src/asset_config.rs
16 burrowland/contracts/contract/src/account.rs
17 burrowland/contracts/contract/src/asset_view.rs
18 burrowland/contracts/contract/src/pyth.rs
19 burrowland/contracts/contract/src/prices.rs
20 burrowland/contracts/contract/src/storage.rs
21 burrowland/contracts/contract/src/shadow_actions.rs
22 burrowland/contracts/contract/src/margin_position.rs
23 burrowland/contracts/contract/src/utils.rs
24 burrowland/contracts/contract/src/margin_base_token_limit.rs
25 burrowland/contracts/contract/src/protocol_debts.rs
26 burrowland/contracts/contract/src/storage_keys.rs
27 burrowland/contracts/contract/src/booster_staking.rs
28 burrowland/contracts/contract/src/actions.rs
29 burrowland/contracts/contract/src/client_echo.rs
30
31 ref-contracts/ref-exchange/src/account_deposit.rs
32 ref-contracts/ref-exchange/src/token_receiver.rs
```

¹https://github.com/burrowHQ/burrowland/tree/margin_trading

²<https://github.com/rhea-finance/burrowland-fix/tree/fix>

³https://github.com/ref-finance/ref-dcl/tree/margin_trading

⁴https://github.com/ref-finance/ref-contracts/tree/margin_trading

```

33
34 ref-dcl-lending/contracts/dcl/src/user_asset.rs
35 ref-dcl-lending/contracts/dcl/src/dcl/utills.rs
36 ref-dcl-lending/contracts/dcl/src/api/token_receiver.rs

```

Listing 1.1: Audit Scope for this Report

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version ([Version 1](#)), as well as new code (in the following versions) to fix issues in the audit report.

Note that the contracts were compiled into WebAssembly (Wasm) binaries in a Docker environment targeting [linux/amd64](#). The code hash was obtained by computing the SHA-256 digest of each Wasm binary and encoding the resulting 32-byte digest in Base58. The resulting code hashes for the Burrowland and Ref Exchange contracts are [6Nek1MZ8cwJJ2xMQAUWup9V5AT-oYS9rkTxJfwo6BC-pFa](#) and [25eNFdfK76NUEZn4cDgq1xQYKjx-KCuzgXmPB3SHBRW4Y](#), respectively.

Project	Version	Commit Hash
Burrowland	Version 1	74462d7e2a299acc0b9702ca278926614c4f4cc8
	Version 2	bfa0b8b75d2d5d978729b22411d44cbdee2156eb
	Version 3	943ea56c7b47856757d57fc18face46d64d8f192
	Version 4	aaaf26979e94617027e9ba72a5c590a498778408
	Version 5	ca24ef55c4fb635525f21646eb788689bf17864a
	Version 6	5a3e2d5331b03e68e692030bf81db4bb7c0871e8
	Version 7	933b0aab9aadd843e8e240cf37f463c0f0eb427d
	Version 8	7d3b8c23402ca240c6ae2bde601ab628249adf93
	Version 9	03d83c3907492c03515f833d988138ce51519317
Ref Exchange	Version 1	9c3797aecf58f0f210ebe73b28b9552345e431f7
	Version 2	76f84f9436b4f6fb4f225ccabd803da31cce521c
Ref Dcl	Version 1	70fbc5b70685afc52113636e5154e5dbfd414b65

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Software Security

- * Reentrancy
- * DoS
- * Access control
- * Data handling and data flow
- * Exception handling
- * Untrusted external call and control flow
- * Initialization consistency
- * Events operation
- * Error-prone randomness
- * Improper use of the proxy system

1.3.2 DeFi Security

- * Semantic consistency
- * Functionality consistency
- * Permission management
- * Business logic
- * Token operation
- * Emergency mechanism
- * Oracle security
- * Whitelist and blacklist
- * Economic impact
- * Batch transfer

1.3.3 NFT Security

- * Duplicated item
- * Verification of the token receiver
- * Off-chain metadata security

1.3.4 Additional Recommendation

- * Gas optimization
- * Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology ⁵ and Common Weakness Enumeration ⁶. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following four categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.

⁵https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

⁶<https://cwe.mitre.org/>

- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Fixed** The item has been confirmed and fixed by the client.

Chapter 2 Findings

In total, we found **fourteen** potential security issues. Besides, we have **five** recommendations and **two** notes.

- High Risk: 6
- Medium Risk: 1
- Low Risk: 7
- Recommendation: 5
- Note: 2

ID	Severity	Description	Category	Status
1	Low	Failure to remove position timestamp from <code>position_latest_actions</code>	Software Security	Fixed
2	High	Lack of sender account check when handling <code>SwapReference</code> message	DeFi Security	Fixed
3	High	Lack of lock when decreasing collateral	DeFi Security	Fixed
4	High	Incorrect enumeration of tokens requiring price feeds	DeFi Security	Fixed
5	Low	Unreasonable <code>pos_id</code> design	DeFi Security	Fixed
6	Low	Lack of reasonable configuration check	DeFi Security	Fixed
7	High	Potential panic during handling message <code>SwapReference</code>	DeFi Security	Fixed
8	Low	Unreasonable check of reserves	DeFi Security	Fixed
9	Low	Potential sandwich attack in force close position token swap	DeFi Security	Confirmed
10	Medium	Unreasonable leverage rate computation	DeFi Security	Fixed
11	Low	Potential gas waste due to improper fee validation	DeFi Security	Fixed
12	High	Lack of account lock check in withdrawal and collateral operations	DeFi Security	Fixed
13	Low	Potential underflow in function <code>internal_storage_balance_of()</code> after forced storage adjustment	DeFi Security	Fixed
14	High	Incorrect aggregation of intermediate swap outputs	DeFi Security	Fixed
15	-	Automatically construct swap indication from the token information	Recommendation	Confirmed
16	-	Use <code>UnorderedMap</code> for <code>margin_positions</code> instead of <code>HashMap</code>	Recommendation	Fixed
17	-	Incorrect error message in <code>get_token_out()</code>	Recommendation	Fixed

18	-	Lack of check in function <code>update_asset()</code>	Recommendation	Fixed
19	-	Remove redundant code in function <code>internal_decrease_collateral_for_gap()</code>	Recommendation	Fixed
20	-	Potential centralization risks	Note	-
21	-	Collateral tracking approximation due to legacy asset accounting	Note	-

The details are provided in the following sections.
software security

2.1 Software Security

2.1.1 Failure to remove position timestamp from `position_latest_actions`

Severity Low

Status Fixed in [Version 4](#)

Introduced by [Version 3](#)

Description The `MarginAccount::position_latest_actions` variable records the timestamp when a position initiates a swap action. This timestamp is meant to be removed once the swap completes, as it is not utilized in storage fee calculations. However, in the `callback_dex_trade()` function, which handles failed swap operations, this timestamp is not deleted. As a result, out-dated timestamps remain in the `position_latest_actions` map, leading to unnecessary storage consumption.

```

725  #[private]
726  pub fn callback_dex_trade(
727      &mut self,
728      account_id: AccountId,
729      pos_id: PosId,
730      amount_in: U128,
731      pre_token_p_amount: U128,
732      op: String,
733  ) {
734      let amount_in_used = if let Some(cross_call_result) = promise_result_as_success() {
735          serde_json::from_slice::<U128>(&cross_call_result)
736              .unwrap()
737              .0
738      } else {
739          0_u128
740      };
741      if amount_in_used == 0 {
742          // trading failed, revert margin operation
743          let mut account = self.internal_unwrap_margin_account(&account_id);
744          if op == "open" {

```

```

745         let mt = account.margin_positions.get(&pos_id).unwrap().clone();
746         let mut asset_d = self.internal_unwrap_asset(&mt.token_d_id);
747         asset_d.margin_pending_debt -= amount_in.0;
748         self.internal_set_asset(&mt.token_d_id, asset_d);
749         account.deposit_supply_shares(&mt.token_c_id, &mt.token_c_shares);
750         // Remove margin_position storage
751         account.storage_tracker.start();
752         account.margin_positions.remove(&pos_id);
753         account.storage_tracker.stop();
754         events::emit::margin_open_failed(&account_id, &pos_id);
755
756     } else if op == "decrease" {
757         let mut mt = account.margin_positions.get(&pos_id).unwrap();
758         let mut asset_p = self.internal_unwrap_asset(&mt.token_p_id);
759         let amount_in: Balance = amount_in.into();
760         let pre_token_p_amount: Balance = pre_token_p_amount.into();
761         if amount_in > pre_token_p_amount {
762             asset_p.margin_position += pre_token_p_amount;
763             // re-deposit those gap to supply as margin
764             let gap = amount_in - pre_token_p_amount;
765             let gap_shares = asset_p.supplied.amount_to_shares(gap, false);
766             asset_p.supplied.deposit(gap_shares, gap);
767             mt.token_c_shares.0 += gap_shares.0;
768         } else {
769             asset_p.margin_position += amount_in;
770         }
771         self.internal_set_asset(&mt.token_p_id, asset_p);
772         mt.is_locking = false;
773         mt.token_p_amount = pre_token_p_amount;
774         // Update existing margin_position storage
775         account.margin_positions.insert(&pos_id, &mt);
776         events::emit::margin_decrease_failed(&account_id, &pos_id);
777     }
778     self.internal_set_margin_account(&account_id, account);
779 }
780 }

```

Listing 2.1: burrowland/contracts/contract/src/margin_position.rs

Impact Failure to remove timestamps from `position_latest_actions` results in outdated data persisting unnecessarily, leading to redundant storage usage.

Suggestion Modify the function `callback_dex_trade()` to ensure the timestamp is properly removed when a swap operation fails.

2.2 DeFi Security

2.2.1 Lack of sender account check when handling SwapReference message

Severity High

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The function `ft_on_transfer()` does not check if the `sender` is a registered `Ref V1` or `Ref V2` contract when handling the `SwapReference` message.

```

88 TokenReceiverMsg::SwapReference { swap_ref } => {
89     let mut account = self.internal_unwrap_margin_account(&swap_ref.account_id);
90     if swap_ref.op == "open" {
91         self.on_open_trade_return(&mut account, amount, &swap_ref);
92     } else if swap_ref.op == "decrease"
93         || swap_ref.op == "close"
94         || swap_ref.op == "liquidate"
95         || swap_ref.op == "forceclose"
96     {
97         let event = self.on_decrease_trade_return(&mut account, amount, &swap_ref);
98         events::emit::margin_decrease_succeeded(&swap_ref.op, event);
99     }
100     self.internal_set_margin_account(&swap_ref.account_id, account);
101     return PromiseOrValue::Value(U128(0));
102 }

```

Listing 2.2: burrowland/contracts/contract/src/fungible_token.rs

Impact An attacker can impersonate a `REF V1` or `REF V2` contract by simply sending the custom `SwapReference` message to the contract. The contract would then correspondingly reduce the user's debt and add assets, while the attacker does not incur any cost.

Suggestion Add a check to ensure that the `sender` must be `REF v1` or `REF V2` contract.

2.2.2 Lack of lock when decreasing collateral

Severity High

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description When a user opens a margin position, the `token_c_shares` value is set appropriately, while the `token_d_shares` and `token_p_amount` values are set to zero. These `token_d_shares` and `token_p_amount` values are intended to be reset upon receiving the `SwapReference` message from the `Ref V1` or `Ref V2` contract. In between these two processes, this margin position is locked.

However, the function `internal_margin_decrease_collateral()` does not check if the margin position is locked. This allows the user to remove almost all of the collateral, putting the margin position in a state where it can be liquidated or force closed.

```

224 pub(crate) fn internal_margin_decrease_collateral(
225     &mut self,
226     account: &mut MarginAccount,
227     pos_id: &PosId,
228     amount: Balance,
229     prices: &Prices,
230 ) -> AccountId {
231     let margin_config = self.internal_margin_config();

```

```
232     let mut mt = account
233         .margin_positions
234         .get(pos_id)
235         .expect("Position not exist")
236         .clone();
237     let token_id = mt.token_c_id.clone();
238     let asset = self.internal_unwrap_asset(&mt.token_c_id);
239     let shares = asset.supplied.amount_to_shares(amount, true);
240
241     // collateral can NOT decrease to 0
242     assert!(
243         mt.token_c_shares.0 > shares.0,
244         "Not enough collateral to decrease"
245     );
246     mt.token_c_shares.0 -= shares.0;
247
248     assert!(
249         !self.is_mt_liquidatable(&mt, prices, margin_config.min_safety_buffer),
250         "Margin position would be below liquidation line"
251     );
252     assert!(
253         !self.is_mt_forcecloseable(&mt, prices),
254         "Margin position would be below forceclose line"
255     );
256
257     assert!(
258         self.get_mtp_lr(&mt, prices).unwrap()
259             <= BigDecimal::from(margin_config.max_leverage_rate as u32),
260         "Leverage rate is too high"
261     );
262
263     account.deposit_supply_shares(&mt.token_c_id, &shares);
264     account.margin_positions.insert(pos_id.clone(), mt);
265
266     token_id
267 }
```

Listing 2.3: burrowland/contracts/contract/src/margin_actions.rs

Impact After opening the position, an attacker can immediately withdraw almost all of the collateral and trigger the swap action of the contract. Despite having slippage protection, there is still some room for a sandwich attack during this swap.

When [Ref V1](#) or [Ref V2](#) returns the swap result, the margin position may be in force-closable state and need to be closed using the contract's [reserves](#), resulting in a loss of the contract's assets.

Suggestion Add a check to ensure that the margin position is not locked before operating in the function [internal_margin_decrease_collateral](#).

2.2.3 Incorrect enumeration of tokens requiring price feeds

Severity High

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The function `margin_involved_tokens()` is intended to enumerate the tokens that require price feeds for a given `MarginActions`. It adds the `token_c_id` and `token_d_id` of the relevant margin positions. However, according to the design, the `token_c_id` may be the same as the `token_d_id`. In this case, the `token_p_id`, whose price is needed for calculating slippage protection, is not included in this enumeration.

```

97  pub fn margin_involved_tokens(&self, account: &MarginAccount, actions: &Vec<MarginAction>) ->
    Vec<TokenId> {
98      let mut tokens = HashSet::new();
99      actions.iter().for_each(|action|{
100          let pos_id = match action {
101              MarginAction::DecreaseCollateral { pos_id, amount: _ } => {
102                  Some(pos_id)
103              }
104              MarginAction::OpenPosition { token_c_id, token_c_amount: _, token_d_id,
                  token_d_amount: _, token_p_id: _, min_token_p_amount: _, swap_indication: _ }
                  => {
105                  tokens.insert(token_c_id.clone());
106                  tokens.insert(token_d_id.clone());
107                  None
108              },
109              MarginAction::DecreaseMTPosition { pos_id, token_p_amount: _, min_token_d_amount: _
                  , swap_indication: _ }=> {
110                  Some(pos_id)
111              }
112              MarginAction::CloseMTPosition { pos_id, token_p_amount: _, min_token_d_amount: _,
                  swap_indication: _ } => {
113                  Some(pos_id)
114              }
115              MarginAction::LiquidateMTPosition { pos_owner_id, pos_id, token_p_amount: _,
                  min_token_d_amount: _, swap_indication: _ } => {
116                  let pos_owner_account = self.internal_get_margin_account(pos_owner_id).expect("
                      Margin account not exist");
117                  let mt = pos_owner_account.margin_positions.get(pos_id).expect("Position not
                      exist");
118                  tokens.insert(mt.token_c_id.clone());
119                  tokens.insert(mt.token_d_id.clone());
120                  None
121              }
122              MarginAction::ForceCloseMTPosition { pos_owner_id, pos_id, token_p_amount: _,
                  min_token_d_amount: _, swap_indication: _ } => {
123                  let pos_owner_account = self.internal_get_margin_account(pos_owner_id).expect("
                      Margin account not exist");
124                  let mt = pos_owner_account.margin_positions.get(pos_id).expect("Position not
                      exist");
125                  tokens.insert(mt.token_c_id.clone());
126                  tokens.insert(mt.token_d_id.clone());
127                  None
128              }
129          _ => None

```

```

130     };
131     if let Some(pos_id) = pos_id {
132         let mt = account.margin_positions.get(pos_id).expect("Position not exist");
133         tokens.insert(mt.token_c_id.clone());
134         tokens.insert(mt.token_d_id.clone());
135     }
136     });
137     tokens.into_iter().collect()
138 }

```

Listing 2.4: burrowland/contracts/contract/src/margin_pyth.rs

Impact The callback function `callback_margin_execute_with_pyth()` that is invoked after `internal_margin_execute_with_pyth()` may fail due to missing price feeds for certain tokens required by the margin action.

Suggestion Modify the function `margin_involved_tokens()` to replace `token_c_id` with `token_p_id`, so that it adds the position token (`token_p_id`) and debt token (`token_d_id`) to the list of tokens requiring price feeds.

2.2.4 Unreasonable pos_id design

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The `pos_id` is generated by concatenating the `account_id` with the `env::block_timestamp()`. This means that within the same block, all generated `pos_ids` with the same `margin` account will be identical.

```

174 pub(crate) fn internal_margin_open_position(
175     &mut self,
176     ts: Timestamp,
177     account: &mut MarginAccount,
178     token_c_id: &AccountId,
179     token_c_amount: Balance,
180     token_d_id: &AccountId,
181     token_d_amount: Balance,
182     token_p_id: &AccountId,
183     min_token_p_amount: Balance,
184     swap_indication: &SwapIndication,
185     prices: &Prices,
186 ) -> EventDataMarginOpen {
187     let pos_id = format!("{}", account.account_id.clone(), ts);
188     assert!(
189         !account.margin_positions.contains_key(&pos_id),
190         "Margin position already exist"
191     );
192
193     let asset_c = self.internal_unwrap_asset(token_c_id);
194     let asset_p = self.internal_unwrap_asset(token_p_id);
195     let mut asset_d = self.internal_unwrap_asset(token_d_id);

```

```
196     let margin_config = self.internal_margin_config();
197
198     // check legitimacy: assets legal; swap_indication matches;
199     margin_config.check_pair(&token_d_id, &token_p_id, &token_c_id);
200     let mut swap_detail = self.parse_swap_indication(swap_indication);
201     let ft_d_amount = token_d_amount / 10u128.pow(asset_d.config.extra_decimals as u32);
202     assert!(
203         swap_detail.verify_token_in(token_d_id, ft_d_amount),
204         "token_in check failed"
205     );
206     let ft_p_amount =
207         min_token_p_amount / 10u128.pow(asset_p.config.extra_decimals as u32);
208     assert!(
209         swap_detail.verify_token_out(token_p_id, ft_p_amount),
210         "token_out check failed"
211     );
212
213     // check safty:
214     // min_position_amount reasonable
215     assert!(
216         is_min_amount_out_reasonable(
217             token_d_amount,
218             &asset_d,
219             prices.get_unwrap(&token_d_id),
220             &asset_p,
221             prices.get_unwrap(&token_p_id),
222             min_token_p_amount,
223             margin_config.max_slippage_rate,
224         ),
225         "min_position_amount is too low"
226     );
227     // margin_hf more than 1 + safty_buffer_rate(10%)
228     let mut mt = MarginTradingPosition::new(
229         ts,
230         token_c_id.clone(),
231         asset_c.supplied.amount_to_shares(token_c_amount, false),
232         token_d_id.clone(),
233         token_p_id.clone(),
234     );
235     mt.token_d_shares = asset_d.margin_debt.amount_to_shares(token_d_amount, true);
236     mt.token_p_amount = min_token_p_amount;
237     assert!(
238         !self.is_mt_liquidatable(&mt, prices, margin_config.min_safty_buffer),
239         "Debt is too much"
240     );
241     assert!(
242         !self.is_mt_forcecloseable(&mt, prices),
243         "Debt is too much"
244     );
245     // leverage rate less than max leverage rate
246     assert!(
247         self.get_mtp_lr(&mt, prices).unwrap()
248         <= BigDecimal::from(margin_config.max_leverage_rate as u32),
```



```
249         "Leverage rate is too high"
250     );
251
252     // passes all check, start to open
253     let event = EventDataMarginOpen {
254         account_id: account.account_id.clone(),
255         pos_id: pos_id.clone(),
256         token_c_id: token_c_id.clone(),
257         token_c_amount,
258         token_c_shares: mt.token_c_shares,
259         token_d_id: token_d_id.clone(),
260         token_d_amount,
261         token_p_id: token_p_id.clone(),
262         token_p_amount: min_token_p_amount,
263     };
264     account.withdraw_supply_shares(token_c_id, &mt.token_c_shares);
265     mt.token_d_shares.0 = 0;
266     mt.token_p_amount = 0;
267     asset_d.increase_margin_pending_debt(token_d_amount, margin_config.pending_debt_scale);
268     self.internal_set_asset(token_d_id, asset_d);
269     // TODO: may need to change to store in an unorderedmap in user Account
270     account.margin_positions.insert(pos_id.clone(), mt);
271
272     // step 4: call dex to trade and wait for callback
273     // organize swap action
274     let swap_ref = SwapReference {
275         account_id: account.account_id.clone(),
276         pos_id: pos_id.clone(),
277         amount_in: token_d_amount.into(),
278         op: format!("open"),
279         liquidator_id: None,
280     };
281     swap_detail.set_client_echo(&swap_ref.to_msg_string());
282     let swap_msg = swap_detail.to_msg_string();
283     ext_fungible_token::ext(token_d_id.clone())
284         .with_attached_deposit(1)
285         .with_static_gas(GAS_FOR_FT_TRANSFER_CALL)
286         .ft_transfer_call(
287             swap_indication.dex_id.clone(),
288             U128(ft_d_amount),
289             None,
290             swap_msg,
291         )
292         .then(
293             Self::ext(env::current_account_id())
294                 .with_static_gas(GAS_FOR_FT_TRANSFER_CALL_CALLBACK)
295                 .callback_dex_trade(
296                     account.account_id.clone(),
297                     pos_id.clone(),
298                     token_d_amount.into(),
299                     U128(0),
300                     format!("open"),
301                 ),
```

```

302     );
303     event
304 }

```

Listing 2.5: burrowland/contracts/contract/src/margin_position.rs

Impact Users are unable to open multiple margin positions within the same block, as all the `pos_ids` generated in that block will be identical.

Suggestion Use a more reasonable method to generate the `pos_id`, such as an auto-incrementing variable.

2.2.5 Lack of reasonable configuration check

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The newly introduced `MarginConfig` contains many global parameters for managing margin trading. These global parameters are used to compare against the user-provided parameters, ensuring that the user does not input values that are too large or too small. However, some of these parameters lack the necessary range checks. As shown in the table below:

Parameter	Reasonable Range
<code>max_leverage_rate</code>	bigger than 1
<code>pending_debt_scale</code>	(0, MAX_RATIO)
<code>max_slippage_rate</code>	(0, MAX_RATIO)
<code>min_safety_buffer</code>	(0, MAX_RATIO)
<code>margin_debt_discount_rate</code>	(0, MAX_RATIO)
<code>open_position_fee_rate</code>	(0, MAX_RATIO)

In addition, the variable "safety" is a typo of "safety".

```

69  #[payable]
70  pub fn update_max_leverage_rate(&mut self, max_leverage_rate: u8) {
71      assert_one_yocto();
72      self.assert_owner();
73      let mut mc = self.internal_margin_config();
74      mc.max_leverage_rate = max_leverage_rate;
75      self.margin_config.set(&mc);
76  }
77
78  #[payable]
79  pub fn update_pending_debt_scale(&mut self, pending_debt_scale: u32) {
80      assert_one_yocto();
81      self.assert_owner();
82      let mut mc = self.internal_margin_config();
83      mc.pending_debt_scale = pending_debt_scale;
84      self.margin_config.set(&mc);
85  }
86
87  #[payable]

```

```
88 pub fn update_max_slippage_rate(&mut self, max_slippage_rate: u32) {
89     assert_one_yocto();
90     self.assert_owner();
91     let mut mc = self.internal_margin_config();
92     mc.max_slippage_rate = max_slippage_rate;
93     self.margin_config.set(&mc);
94 }
95
96 #[payable]
97 pub fn update_min_safety_buffer(&mut self, min_safety_buffer: u32) {
98     assert_one_yocto();
99     self.assert_owner();
100    let mut mc = self.internal_margin_config();
101    mc.min_safety_buffer = min_safety_buffer;
102    self.margin_config.set(&mc);
103 }
104
105 #[payable]
106 pub fn update_margin_debt_discount_rate(&mut self, margin_debt_discount_rate: u32) {
107     assert_one_yocto();
108     self.assert_owner();
109     let mut mc = self.internal_margin_config();
110     mc.margin_debt_discount_rate = margin_debt_discount_rate;
111     self.margin_config.set(&mc);
112 }
113
114 #[payable]
115 pub fn update_open_position_fee_rate(&mut self, open_position_fee_rate: u32) {
116     assert_one_yocto();
117     self.assert_owner();
118     let mut mc = self.internal_margin_config();
119     mc.open_position_fee_rate = open_position_fee_rate;
120     self.margin_config.set(&mc);
121 }
```

Listing 2.6: burrowland/contracts/contract/src/margin_config.rs

Impact Unreasonable configuration values due to missing range checks can cause the contract to not work as intended.

Suggestion Add corresponding checks according to the table.

2.2.6 Potential panic during handling message SwapReference

Severity High

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description When handling the [SwapReference](#) message, the transaction should not fail. Otherwise, the user would lose the assets they obtained from the [Ref V1](#) or [Ref V2](#) contract, causing an inconsistency in the contract state. The corresponding margin position would also remain locked and unable to be unlocked. However, during this process, the function

`internal_set_margin_account()` needs to be invoked to write back the account states of the `liquidator` and `account`, which includes storage checks that may cause a panic by insufficient storage fees.

```

88 TokenReceiverMsg::SwapReference { swap_ref } => {
89     let mut account = self.internal_unwrap_margin_account(&swap_ref.account_id);
90     if swap_ref.op == "open" {
91         self.on_open_trade_return(&mut account, amount, &swap_ref);
92     } else if swap_ref.op == "decrease"
93         || swap_ref.op == "close"
94         || swap_ref.op == "liquidate"
95         || swap_ref.op == "forceclose"
96     {
97         let event = self.on_decrease_trade_return(&mut account, amount, &swap_ref);
98         events::emit::margin_decrease_succeeded(&swap_ref.op, event);
99     }
100     self.internal_set_margin_account(&swap_ref.account_id, account);
101     return PromiseOrValue::Value(U128(0));
102 }

```

Listing 2.7: burrowland/contracts/contract/src/fungible_token.rs

```

366 pub(crate) fn on_decrease_trade_return(
367     &mut self,
368     account: &mut MarginAccount,
369     amount: Balance,
370     sr: &SwapReference,
371 ) -> EventDataMarginDecreaseResult {
372     let mut mt = account.margin_positions.get(&sr.pos_id).unwrap().clone();
373     let mut asset_debt = self.internal_unwrap_asset(&mt.token_d_id);
374     let mut asset_position = self.internal_unwrap_asset(&mt.token_p_id);
375     let (mut benefit_m_shares, mut benefit_d_shares, mut benefit_p_shares) =
376         (0_u128, 0_u128, 0_u128);
377
378     // figure out actual repay amount and shares
379     // figure out how many debt_cap been repaid, and charge corresponding holding-position fee
380     // from repayment.
381     let debt_amount = asset_debt
382         .margin_debt
383         .shares_to_amount(mt.token_d_shares, true);
384     let hp_fee = u128_ratio(
385         mt.debt_cap,
386         asset_debt.unit_acc_hp_interest - mt.uahpi_at_open,
387         UNIT,
388     );
389     let repay_cap = u128_ratio(mt.debt_cap, amount, debt_amount + hp_fee);
390     let (repay_amount, repay_shares, left_amount, repay_hp_fee) = if repay_cap >= mt.debt_cap {
391         (debt_amount, mt.token_d_shares, amount - debt_amount - hp_fee, hp_fee)
392     } else {
393         let repay_hp_fee = u128_ratio(hp_fee, repay_cap, mt.debt_cap);
394         (
395             amount - repay_hp_fee,

```

```
396         asset_debt
397             .margin_debt
398             .amount_to_shares(amount - repay_hp_fee, false),
399         0,
400         repay_hp_fee,
401     )
402 };
403 asset_debt.margin_debt.withdraw(repay_shares, repay_amount);
404 mt.token_d_shares.0 -= repay_shares.0;
405 mt.debt_cap = if repay_cap >= mt.debt_cap {
406     0
407 } else {
408     mt.debt_cap - repay_cap
409 };
410 // distribute hp_fee
411 asset_debt.prot_fee += repay_hp_fee;
412
413 // handle possible leftover debt asset, put them into user's supply
414 if left_amount > 0 {
415     let supply_shares = asset_debt.supplied.amount_to_shares(left_amount, false);
416     if supply_shares.0 > 0 {
417         asset_debt.supplied.deposit(supply_shares, left_amount);
418         benefit_d_shares = supply_shares.0;
419     }
420 }
421
422 if sr.op != "decrease" {
423     // try to repay remaining debt from margin
424     if mt.token_d_shares.0 > 0 && mt.token_d_id == mt.token_c_id {
425         let remain_debt_balance = asset_debt
426             .margin_debt
427             .shares_to_amount(mt.token_d_shares, true);
428         let margin_shares_to_repay = asset_debt
429             .supplied
430             .amount_to_shares(remain_debt_balance, true);
431         let (repay_debt_share, used_supply_share, repay_amount) =
432             if margin_shares_to_repay <= mt.token_c_shares {
433                 (mt.token_d_shares, margin_shares_to_repay, remain_debt_balance)
434             } else {
435                 // use all margin balance to repay
436                 let margin_balance = asset_debt
437                     .supplied
438                     .shares_to_amount(mt.token_c_shares, false);
439                 let repay_debt_shares = asset_debt
440                     .margin_debt
441                     .amount_to_shares(margin_balance, false);
442                 (repay_debt_shares, mt.token_c_shares, margin_balance)
443             };
444         asset_debt
445             .supplied
446             .withdraw(used_supply_share, repay_amount);
447         asset_debt
448             .margin_debt
```

```
449         .withdraw(repay_debt_share, repay_amount);
450         mt.token_d_shares.0 -= repay_debt_share.0;
451         mt.token_c_shares.0 -= used_supply_share.0;
452     }
453 }
454
455 if sr.op == "forceclose" {
456     // try to use protocol reserve to repay remaining debt
457     if mt.token_d_shares.0 > 0 {
458         let remain_debt_balance = asset_debt
459             .margin_debt
460             .shares_to_amount(mt.token_d_shares, true);
461         if asset_debt.reserved > remain_debt_balance {
462             asset_debt.reserved -= remain_debt_balance;
463             asset_debt
464                 .margin_debt
465                 .withdraw(mt.token_d_shares, remain_debt_balance);
466             mt.token_d_shares.0 = 0;
467         }
468     }
469 }
470
471 mt.is_locking = false;
472 account
473     .margin_positions
474     .insert(sr.pos_id.clone(), mt.clone());
475
476
477 let event = EventDataMarginDecreaseResult {
478     account_id: account.account_id.clone(),
479     pos_id: sr.pos_id.clone(),
480     liquidator_id: sr.liquidator_id.clone(),
481     token_c_id: mt.token_c_id.clone(),
482     token_c_shares: mt.token_c_shares,
483     token_d_id: mt.token_d_id.clone(),
484     token_d_shares: mt.token_d_shares,
485     token_p_id: mt.token_p_id.clone(),
486     token_p_amount: mt.token_p_amount,
487     holding_fee: repay_hp_fee,
488 };
489
490 // try to settle this position
491 if mt.token_d_shares.0 == 0 {
492     // close this position and remaining asset goes back to user's inner account
493     // TODO: change to directly send assets back to user
494     if mt.token_c_shares.0 > 0 {
495         benefit_m_shares = mt.token_c_shares.0;
496     }
497     if mt.token_p_amount > 0 {
498         let position_shares = asset_position
499             .supplied
500             .amount_to_shares(mt.token_p_amount, false);
501         asset_position
```

```
502         .supplied
503         .deposit(position_shares, mt.token_p_amount);
504         asset_position.margin_position -= mt.token_p_amount;
505         benefit_p_shares = position_shares.0;
506     }
507     account.margin_positions.remove(&sr.pos_id);
508 } else {
509     if sr.op != "decrease" {
510         env::log_str(&format!(
511             "{} failed due to insufficient fund, user {}, pos_id {}",
512             sr.op.clone(),
513             account.account_id.clone(),
514             sr.pos_id.clone()
515         ));
516     }
517 }
518
519 // distribute benefits
520 if benefit_d_shares > 0 || benefit_m_shares > 0 || benefit_p_shares > 0 {
521     if sr.op == "liquidate" || sr.op == "forceclose" {
522         let mut liquidator_account =
523             if let Some(ref liquidator_account_id) = sr.liquidator_id {
524                 if let Some(x) = self.internal_get_margin_account(&liquidator_account_id) {
525                     x
526                 } else {
527                     self.internal_unwrap_margin_account(&self.internal_config().owner_id)
528                 }
529             } else {
530                 self.internal_unwrap_margin_account(&self.internal_config().owner_id)
531             };
532         if benefit_d_shares > 0 {
533             liquidator_account
534                 .deposit_supply_shares(&mt.token_d_id, &U128(benefit_d_shares));
535         }
536         if benefit_m_shares > 0 {
537             liquidator_account
538                 .deposit_supply_shares(&mt.token_c_id, &U128(benefit_m_shares));
539         }
540         if benefit_p_shares > 0 {
541             liquidator_account
542                 .deposit_supply_shares(&mt.token_p_id, &U128(benefit_p_shares));
543         }
544         self.internal_set_margin_account(
545             &liquidator_account.account_id.clone(),
546             liquidator_account,
547         );
548     } else {
549         if benefit_d_shares > 0 {
550             account.deposit_supply_shares(&mt.token_d_id, &U128(benefit_d_shares));
551         }
552         if benefit_m_shares > 0 {
553             account.deposit_supply_shares(&mt.token_c_id, &U128(benefit_m_shares));
554         }
555     }
```

```
555         if benefit_p_shares > 0 {
556             account.deposit_supply_shares(&mt.token_p_id, &U128(benefit_p_shares));
557         }
558     }
559 }
560
561 self.internal_set_asset(&mt.token_d_id, asset_debt);
562 self.internal_set_asset(&mt.token_p_id, asset_position);
563
564 event
565 }
```

Listing 2.8: burrowland/contracts/contract/src/margin_trading.rs

Impact Users may lose the assets they obtained from the [Ref V1](#) or [Ref V2](#) contract, causing an inconsistency in the contract state. The corresponding margin position would also remain locked and unable to be unlocked.

Suggestion Revise the corresponding logic.

2.2.7 Unreasonable check of reserves

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description When executing a force close operation in the function `on_decrease_trade_return()`, any remaining `debt` needs to be covered by the contract's `reserves`. However, when comparing the amount between the contract's `reserves` and the remaining `debt` amount, the check requires the `reserves` to be strictly greater than the remaining `debt`.

```
366 pub(crate) fn on_decrease_trade_return(
367     &mut self,
368     account: &mut MarginAccount,
369     amount: Balance,
370     sr: &SwapReference,
371 ) -> EventDataMarginDecreaseResult {
372     let mut mt = account.margin_positions.get(&sr.pos_id).unwrap().clone();
373     let mut asset_debt = self.internal_unwrap_asset(&mt.token_d_id);
374     let mut asset_position = self.internal_unwrap_asset(&mt.token_p_id);
375     let (mut benefit_m_shares, mut benefit_d_shares, mut benefit_p_shares) =
376         (0_u128, 0_u128, 0_u128);
377
378     // figure out actual repay amount and shares
379     // figure out how many debt_cap been repaid, and charge corresponding holding-position fee
380     // from repayment.
381     let debt_amount = asset_debt
382         .margin_debt
383         .shares_to_amount(mt.token_d_shares, true);
384     let hp_fee = u128_ratio(
385         mt.debt_cap,
```



```
385     asset_debt.unit_acc_hp_interest - mt.uahpi_at_open,
386     UNIT,
387 );
388 let repay_cap = u128_ratio(mt.debt_cap, amount, debt_amount + hp_fee);
389
390 let (repay_amount, repay_shares, left_amount, repay_hp_fee) = if repay_cap >= mt.debt_cap {
391     (debt_amount, mt.token_d_shares, amount - debt_amount - hp_fee, hp_fee)
392 } else {
393     let repay_hp_fee = u128_ratio(hp_fee, repay_cap, mt.debt_cap);
394     (
395         amount - repay_hp_fee,
396         asset_debt
397             .margin_debt
398             .amount_to_shares(amount - repay_hp_fee, false),
399         0,
400         repay_hp_fee,
401     )
402 };
403 asset_debt.margin_debt.withdraw(repay_shares, repay_amount);
404 mt.token_d_shares.0 -= repay_shares.0;
405 mt.debt_cap = if repay_cap >= mt.debt_cap {
406     0
407 } else {
408     mt.debt_cap - repay_cap
409 };
410 // distribute hp_fee
411 asset_debt.prot_fee += repay_hp_fee;
412
413 // handle possible leftover debt asset, put them into user's supply
414 if left_amount > 0 {
415     let supply_shares = asset_debt.supplied.amount_to_shares(left_amount, false);
416     if supply_shares.0 > 0 {
417         asset_debt.supplied.deposit(supply_shares, left_amount);
418         benefit_d_shares = supply_shares.0;
419     }
420 }
421
422 if sr.op != "decrease" {
423     // try to repay remaining debt from margin
424     if mt.token_d_shares.0 > 0 && mt.token_d_id == mt.token_c_id {
425         let remain_debt_balance = asset_debt
426             .margin_debt
427             .shares_to_amount(mt.token_d_shares, true);
428         let margin_shares_to_repay = asset_debt
429             .supplied
430             .amount_to_shares(remain_debt_balance, true);
431         let (repay_debt_share, used_supply_share, repay_amount) =
432             if margin_shares_to_repay <= mt.token_c_shares {
433                 (mt.token_d_shares, margin_shares_to_repay, remain_debt_balance)
434             } else {
435                 // use all margin balance to repay
436                 let margin_balance = asset_debt
437                     .supplied
```

```
438         .shares_to_amount(mt.token_c_shares, false);
439         let repay_debt_shares = asset_debt
440             .margin_debt
441             .amount_to_shares(margin_balance, false);
442         (repay_debt_shares, mt.token_c_shares, margin_balance)
443     };
444     asset_debt
445         .supplied
446         .withdraw(used_supply_share, repay_amount);
447     asset_debt
448         .margin_debt
449         .withdraw(repay_debt_share, repay_amount);
450     mt.token_d_shares.0 -= repay_debt_share.0;
451     mt.token_c_shares.0 -= used_supply_share.0;
452 }
453 }
454
455 if sr.op == "forceclose" {
456     // try to use protocol reserve to repay remaining debt
457     if mt.token_d_shares.0 > 0 {
458         let remain_debt_balance = asset_debt
459             .margin_debt
460             .shares_to_amount(mt.token_d_shares, true);
461         if asset_debt.reserved > remain_debt_balance {
462             asset_debt.reserved -= remain_debt_balance;
463             asset_debt
464                 .margin_debt
465                 .withdraw(mt.token_d_shares, remain_debt_balance);
466             mt.token_d_shares.0 = 0;
467         }
468     }
469 }
470
471 mt.is_locking = false;
472 account
473     .margin_positions
474     .insert(sr.pos_id.clone(), mt.clone());
475
476
477 let event = EventDataMarginDecreaseResult {
478     account_id: account.account_id.clone(),
479     pos_id: sr.pos_id.clone(),
480     liquidator_id: sr.liquidator_id.clone(),
481     token_c_id: mt.token_c_id.clone(),
482     token_c_shares: mt.token_c_shares,
483     token_d_id: mt.token_d_id.clone(),
484     token_d_shares: mt.token_d_shares,
485     token_p_id: mt.token_p_id.clone(),
486     token_p_amount: mt.token_p_amount,
487     holding_fee: repay_hp_fee,
488 };
489
490 // try to settle this position
```

```
491     if mt.token_d_shares.0 == 0 {
492         // close this position and remaining asset goes back to user's inner account
493         // TODO: change to directly send assets back to user
494         if mt.token_c_shares.0 > 0 {
495             benefit_m_shares = mt.token_c_shares.0;
496         }
497         if mt.token_p_amount > 0 {
498             let position_shares = asset_position
499                 .supplied
500                 .amount_to_shares(mt.token_p_amount, false);
501             asset_position
502                 .supplied
503                 .deposit(position_shares, mt.token_p_amount);
504             asset_position.margin_position -= mt.token_p_amount;
505             benefit_p_shares = position_shares.0;
506         }
507         account.margin_positions.remove(&sr.pos_id);
508     } else {
509         if sr.op != "decrease" {
510             env::log_str(&format!(
511                 "{} failed due to insufficient fund, user {}, pos_id {}",
512                 sr.op.clone(),
513                 account.account_id.clone(),
514                 sr.pos_id.clone()
515             ));
516         }
517     }
518
519     // distribute benefits
520     if benefit_d_shares > 0 || benefit_m_shares > 0 || benefit_p_shares > 0 {
521         if sr.op == "liquidate" || sr.op == "forceclose" {
522             let mut liquidator_account =
523                 if let Some(ref liquidator_account_id) = sr.liquidator_id {
524                     if let Some(x) = self.internal_get_margin_account(&liquidator_account_id) {
525                         x
526                     } else {
527                         self.internal_unwrap_margin_account(&self.internal_config().owner_id)
528                     }
529                 } else {
530                     self.internal_unwrap_margin_account(&self.internal_config().owner_id)
531                 };
532             if benefit_d_shares > 0 {
533                 liquidator_account
534                     .deposit_supply_shares(&mt.token_d_id, &U128(benefit_d_shares));
535             }
536             if benefit_m_shares > 0 {
537                 liquidator_account
538                     .deposit_supply_shares(&mt.token_c_id, &U128(benefit_m_shares));
539             }
540             if benefit_p_shares > 0 {
541                 liquidator_account
542                     .deposit_supply_shares(&mt.token_p_id, &U128(benefit_p_shares));
543             }
544         }
545     }
```

```
544         self.internal_set_margin_account(  
545             &liquidator_account.account_id.clone(),  
546             liquidator_account,  
547         );  
548     } else {  
549         if benefit_d_shares > 0 {  
550             account.deposit_supply_shares(&mt.token_d_id, &U128(benefit_d_shares));  
551         }  
552         if benefit_m_shares > 0 {  
553             account.deposit_supply_shares(&mt.token_c_id, &U128(benefit_m_shares));  
554         }  
555         if benefit_p_shares > 0 {  
556             account.deposit_supply_shares(&mt.token_p_id, &U128(benefit_p_shares));  
557         }  
558     }  
559 }  
560  
561 self.internal_set_asset(&mt.token_d_id, asset_debt);  
562 self.internal_set_asset(&mt.token_p_id, asset_position);  
563  
564 event  
565 }
```

Listing 2.9: burrowland/contracts/contract/src/margin_trading.rs

Impact Margin positions that can be forced closed may fail to be closed.

Suggestion Revise the check to be greater or equal.

2.2.8 Potential sandwich attack in force close position token swap

Severity Low

Status Confirmed

Introduced by [Version 1](#)

Description The force closing margin position operation swaps the liquidated user's `position` token into the `debt` token. Despite having slippage protection, there is still some room for a sandwich attack during this swap.

This issue is similar to issue -2, where both involve a potential sandwich attack on a token swap. However, issue -2 can be reliably triggered by an attacker, whereas under normal circumstances, the probability of a margin position reaching a force-closable state to trigger this swap is relatively low.

```
307 pub(crate) fn process_decrease_margin_position(  
308     &mut self,  
309     account: &mut MarginAccount,  
310     pos_id: &String,  
311     token_p_amount: Balance,  
312     min_token_d_amount: Balance,  
313     swap_indication: &SwapIndication,  
314     prices: &Prices,  
315     op: String,
```

```
316     liquidator_id: Option<AccountId>,
317 ) -> EventDataMarginDecrease {
318     let mut mt = account
319         .margin_positions
320         .get_mut(pos_id)
321         .expect("Position not exist");
322     assert!(
323         !mt.is_locking,
324         "Position is currently waiting for a trading result."
325     );
326     let pre_token_p_amount = mt.token_p_amount;
327     let mut asset_p = self.internal_unwrap_asset(&mt.token_p_id);
328     let asset_d = self.internal_unwrap_asset(&mt.token_d_id);
329     let margin_config = self.internal_margin_config();
330
331     // check swap_indication
332     let mut swap_detail = self.parse_swap_indication(swap_indication);
333     let ft_p_amount =
334         token_p_amount / 10u128.pow(asset_p.config.extra_decimals as u32);
335     assert!(
336         swap_detail.verify_token_in(&mt.token_p_id, ft_p_amount),
337         "token_in check failed"
338     );
339     let ft_d_amount = min_token_d_amount / 10u128.pow(asset_d.config.extra_decimals as u32);
340     assert!(
341         swap_detail.verify_token_out(&mt.token_d_id, ft_d_amount),
342         "token_out check failed"
343     );
344
345     // min_debt_amount reasonable
346     assert!(
347         is_min_amount_out_reasonable(
348             token_p_amount,
349             &asset_p,
350             prices.get_unwrap(&mt.token_p_id),
351             &asset_d,
352             prices.get_unwrap(&mt.token_d_id),
353             min_token_d_amount,
354             margin_config.max_slippage_rate,
355         ),
356         "min_debt_amount is too low"
357     );
358
359     if op == "close" || op == "liquidate" {
360         // ensure all debt would be repaid
361         // and take holding-position fee into account
362         let total_debt_amount = asset_d
363             .margin_debt
364             .shares_to_amount(mt.token_d_shares, true);
365         let hp_fee = u128_ratio(
366             mt.debt_cap,
367             asset_d.unit_acc_hp_interest - mt.uahpi_at_open,
368             UNIT,
```

```
369     );
370     if min_token_d_amount < total_debt_amount + hp_fee {
371         assert_eq!(
372             mt.token_c_id, mt.token_d_id,
373             "Can NOT trade under total debt when margin and debt asset are not the same"
374         );
375         let gap_shares = asset_d
376             .supplied
377             .amount_to_shares(total_debt_amount + hp_fee - min_token_d_amount, true);
378         assert!(
379             mt.token_c_shares.0 > gap_shares.0,
380             "Not all debt could be repaid"
381         );
382     }
383 }
384
385 if op == "liquidate" {
386     assert!(
387         self.is_mt_liquidatable(&mt, prices, margin_config.min_safety_buffer),
388         "Margin position is not liquidatable"
389     );
390 } else if op == "forceclose" {
391     assert!(
392         self.is_mt_forcecloseable(&mt, prices),
393         "Margin position is not forceclose-able"
394     );
395 }
396
397 // ensure enough position token to trade
398 if token_p_amount > mt.token_p_amount {
399     // try to add some of margin asset into trading
400     assert_eq!(
401         mt.token_c_id, mt.token_p_id,
402         "Not enough position asset balance"
403     );
404     let gap_shares = asset_p
405         .supplied
406         .amount_to_shares(token_p_amount - mt.token_p_amount, true);
407     mt.token_c_shares
408         .0
409         .checked_sub(gap_shares.0)
410         .expect("Not enough position asset balance");
411     asset_p
412         .supplied
413         .withdraw(gap_shares, token_p_amount - mt.token_p_amount);
414     asset_p.margin_position -= mt.token_p_amount;
415     mt.token_p_amount = 0;
416 } else {
417     asset_p.margin_position -= token_p_amount;
418     mt.token_p_amount -= token_p_amount;
419 }
420
421 // prepare to close
```

```

422     mt.is_locking = true;
423     self.internal_set_asset(&mt.token_p_id, asset_p);
424     // TODO: mt may be needed to change to store in an unorderedmap in user Account
425
426     let event = EventDataMarginDecrease {
427         account_id: account.account_id.clone(),
428         pos_id: pos_id.clone(),
429         liquidator_id: liquidator_id.clone(),
430         token_p_id: mt.token_p_id.clone(),
431         token_p_amount,
432         token_d_id: mt.token_d_id.clone(),
433         token_d_amount: min_token_d_amount,
434     };
435
436     // step 3: call dex to trade and wait for callback
437     // organize swap action
438     let swap_ref = SwapReference {
439         account_id: account.account_id.clone(),
440         pos_id: pos_id.clone(),
441         amount_in: token_p_amount.into(),
442         op,
443         liquidator_id,
444     };
445     swap_detail.set_client_echo(&swap_ref.to_msg_string());
446     let swap_msg = swap_detail.to_msg_string();
447     ext_fungible_token::ext(mt.token_p_id.clone())
448         .with_attached_deposit(1)
449         .with_static_gas(GAS_FOR_FT_TRANSFER_CALL)
450         .ft_transfer_call(
451             swap_indication.dex_id.clone(),
452             U128(ft_p_amount),
453             None,
454             swap_msg,
455         )
456         .then(
457             Self::ext(env::current_account_id())
458                 .with_static_gas(GAS_FOR_FT_TRANSFER_CALL_CALLBACK)
459                 .callback_dex_trade(
460                     account.account_id.clone(),
461                     pos_id.clone(),
462                     token_p_amount.into(),
463                     pre_token_p_amount.into(),
464                     format!("decrease"),
465                 ),
466         );
467     event
468 }

```

Listing 2.10: burrowland/contracts/contract/src/margin_position.rs

Impact An attacker can set the `min_amount_d_out` to be as small as possible within the allowed slippage protection range. This enables them to conduct a sandwich attack, forcing the contract to use more of its reserves to cover the liquidation.

Suggestion The value of the parameter `max_slippage_rate` for slippage protection should be set more strictly (smaller) when it comes to force closing, distinguishing it from the values used for opening or other operations.

Feedback from the Project Accept this situation and try to ensure success when the force closing is initiated.

2.2.9 Unreasonable leverage rate computation

Severity Medium

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The hold position fee is not included in leverage rate computation in the function `get_mtp_lr`.

```
160 pub(crate) fn get_mtp_lr(  
161     &self,  
162     mt: &MarginTradingPosition,  
163     prices: &Prices,  
164 ) -> Option<BigDecimal> {  
165     if mt.token_c_shares.0 == 0 || mt.token_d_shares.0 == 0 {  
166         None  
167     } else {  
168         Some(self.get_mtp_debt_value(&mt, prices) / self.get_mtp_collateral_value(&mt, prices))  
169     }  
170 }
```

Listing 2.11: burrowland/contracts/contract/src/margin_position.rs

Impact The computation of the leverage rate is inaccurate.

Suggestion Add the hold position fee into the computation of the leverage rate.

2.2.10 Potential gas waste due to improper fee validation

Severity Low

Status Fixed in [Version 5](#)

Introduced by [Version 4](#)

Description The protocol allows beneficiaries to withdraw their accumulated `beneficiary_fee` through the function `withdraw_beneficiary_fee()`. Since multiple tokens are supported, the protocol applies `extra_decimals` for internal accounting by scaling token amounts, and later rescales them during withdrawal.

Within the function, the existence of accrued fees is determined by checking whether `stdd_amount` is greater than zero. However, after rescaling, the resulting `ft_amount` may equal zero even when `stdd_amount > 0`. In such cases, the call to function `ft_transfer()` will revert due to its internal validation that transferred amounts must be greater than zero. This leads to unnecessary gas consumption without achieving any meaningful effect.


```
498 pub fn withdraw_beneficiary_fee(&mut self, token_id: AccountId) -> PromiseOrValue<bool> {
499     assert_one_yocto();
500     let beneficiary = env::predecessor_account_id();
501     let mut asset = self.internal_unwrap_asset(&token_id);
502
503     let stdd_amount = asset.beneficiary_fees.get(&beneficiary).unwrap_or(&U128(0)).0;
504
505     if stdd_amount > 0 {
506         asset.beneficiary_fees.remove(&beneficiary);
507         let ft_amount = stdd_amount / 10u128.pow(asset.config.extra_decimals as u32);
508         self.internal_set_asset(&token_id, asset);
509         events::emit::withdraw_beneficiary_fee_started(&beneficiary, stdd_amount, &token_id);
510         self.internal_beneficiary_withdraw(&beneficiary, &token_id, stdd_amount, ft_amount).
            into()
511     } else {
512         PromiseOrValue::Value(false)
513     }
514 }
```

Listing 2.12: burrowland/contracts/contract/src/config.rs

Impact Users may incur unnecessary gas costs when withdrawals revert due to zero `ft_amount`.

Suggestion Perform validation after rescaling and ensure that `ft_amount` is greater than zero before proceeding with withdrawal.

2.2.11 Lack of account lock check in withdrawal and collateral operations

Severity High

Status Fixed in [Version 6](#)

Introduced by [Version 5](#)

Description The function `simple_withdraw()` allows users to withdraw assets they have deposited into the protocol. When the user's internal balance is insufficient to cover the requested withdrawal amount, the function may partially unwind the user's position to release additional collateral.

During this process, the protocol performs a cross-contract call to the [Pyth](#) oracle to fetch real-time prices of involved tokens, and then proceeds with the collateralization calculation in the callback function `callback_simple_withdraw_with_pyth()`.

However, neither function `simple_withdraw()` nor function `callback_simple_withdraw_with_pyth()` verifies whether the account is currently locked. The protocol's liquidation-related flows (e.g., `shadow_liquidate` and `force_close`) rely on account-level locking to prevent concurrent state changes while cross-contract operations such as swaps are pending.

By skipping this lock check, a user may still execute withdrawals during liquidation or forced closure, potentially leading to inconsistent account states and undermining liquidation safety guarantees.

The same issue also exists in functions `internal_only_increase_collateral()` and `internal_only_repay()`.

```

833  #[payable]
834  pub fn simple_withdraw(&mut self, token_id: AccountId, amount_with_inner_decimal: U128,
    recipient_id: Option<AccountId>) -> Promise {
835      assert_one_yocto();
836
837      let account_id = env::predecessor_account_id();
838      let recipient_id = recipient_id.unwrap_or(account_id.clone());
839      let mut account = self.internal_unwrap_account(&account_id);
840
841      assert!(!token_id.to_string().starts_with(SHADOW_V1_TOKEN_PREFIX));
842
843      // check if has enough token in user's supply
844      let mut gap = amount_with_inner_decimal.0;
845      if let Some(shares) = account.supplied.get(&token_id) {
846          let asset = self.internal_unwrap_asset(&token_id);
847          let amount = asset.supplied.shares_to_amount(shares.clone(), false);
848          gap = gap.saturating_sub(amount);
849      }
850
851      if gap > 0 {
852          // try use collateral asset
853          if let Some(pyth_promise) = self.internal_decrease_collateral_for_gap(&account_id, &mut
            account, &token_id, amount_with_inner_decimal.0, gap, &recipient_id) {
854              return pyth_promise;
855          }
856      }
857
858      // finally withdraw from supply
859      let promise = self.internal_simple_withdraw(&account_id, &mut account, &token_id,
            amount_with_inner_decimal.0, &recipient_id);
860      self.internal_set_account(&account_id, account);
861      promise
862  }

```

Listing 2.13: burrowland/contracts/contract/src/actions.rs

```

936  pub fn internal_decrease_collateral_for_gap(
937      &mut self,
938      account_id: &AccountId,
939      account: &mut Account,
940      token_id: &AccountId,
941      total_amount: u128,
942      gap_amount: u128,
943      recipient_id: &AccountId,
944  ) -> Option<Promise> {
945      let actions = vec![Action::DecreaseCollateral(AssetAmount {
946          token_id: token_id.clone(),
947          amount: Some(gap_amount.into()),
948          max_amount: None
949      })];
950      let involved_tokens: Vec<AccountId> = self.involved_tokens(&account, &actions);
951      if involved_tokens.len() > 0 {
952          assert!(self.internal_config().enable_pyth_oracle, "Pyth oracle disabled");

```

```

953     let (promise_token_ids, default_prices) = self.prepare_promise_tokens(&involved_tokens)
954     ;
955     if promise_token_ids.len() > 0 {
956         let (all_promise_flags, promise) = self.generate_flags_and_promise(&
957             promise_token_ids);
958         let callback_gas = env::prepaid_gas() - (GAS_FOR_GET_PRICE) * all_promise_flags.len
959             () as u64 - GAS_FOR_BUFFER;
960         Some(promise.then(
961             Self::ext(env::current_account_id())
962             .with_static_gas(callback_gas)
963             .callback_simple_withdraw_with_pyth(
964                 account_id.clone(),
965                 involved_tokens,
966                 all_promise_flags,
967                 token_id.clone(),
968                 total_amount.into(),
969                 gap_amount.into(),
970                 recipient_id.clone(),
971                 default_prices)
972         ))
973     } else {
974         self.internal_simple_decrease_collateral(
975             account_id,
976             account,
977             token_id,
978             gap_amount,
979             Prices::from_prices(default_prices));
980         None
981     }
982 } else {
983     self.internal_simple_decrease_collateral(
984         account_id,
985         account,
986         token_id,
987         gap_amount,
988         Prices::new());
989     None
990 }
991 }

```

Listing 2.14: burrowland/contracts/contract/src/actions.rs

```

864     #[private]
865     pub fn callback_simple_withdraw_with_pyth(
866         &mut self,
867         account_id: AccountId,
868         involved_tokens: Vec<TokenId>,
869         all_promise_flags: Vec<String>,
870         token_id: AccountId,
871         total_amount: U128,
872         gap_amount: U128,
873         recipient_id: AccountId,
874         default_prices: HashMap<TokenId, Price>,

```

```
875 ) -> Promise {
876     assert!(env::promise_results_count() == all_promise_flags.len() as u64, "Invalid promise
      count");
877     let all_prices = self.generate_all_prices(involved_tokens, all_promise_flags,
      default_prices);
878     let mut account = self.internal_unwrap_account(&account_id);
879
880     self.internal_simple_decrease_collateral(&account_id, &mut account, &token_id, gap_amount
      .0, all_prices);
881     let promise = self.internal_simple_withdraw(&account_id, &mut account, &token_id,
      total_amount.0, &recipient_id);
882     self.internal_set_account(&account_id, account);
883     promise
884 }
```

Listing 2.15: burrowland/contracts/contract/src/actions.rs

```
5 pub fn internal_only_increase_collateral(
6     &mut self,
7     account_id: &AccountId,
8     token_id: &AccountId,
9     amount: Balance, // inner decimal precision
10 ) {
11     // 1. check stage
12     let asset = self.internal_unwrap_asset(&token_id);
13     assert!(
14         asset.config.can_use_as_collateral,
15         "This asset can't be used as a collateral"
16     );
17     // check if supply limit has hit, then need panic here
18     if let Some(supplied_limit) = asset.config.supplied_limit {
19         assert!(
20             asset.supplied.balance + amount <= supplied_limit.0,
21             "Asset {} has hit supply limit, increasing collateral is not allowed", token_id
22         );
23     }
24     let shares: Shares = asset.supplied.amount_to_shares(amount, false);
25
26     // 2. on asset level, supply (including collateral) increased
27     let mut asset = self.internal_unwrap_asset(token_id);
28     asset.supplied.deposit(shares, amount);
29
30     // 3. on account level, supply (excluding collateral) untouched but position changed
31     let mut account = self.internal_unwrap_account(account_id);
32     // account_asset untouched cause no pure supply change, but farms affected
33     account.add_affected_farm(FarmId::Supplied(token_id.clone()));
34     account.add_affected_farm(FarmId::TokenNetBalance(token_id.clone()));
35     let position = REGULAR_POSITION.to_string();
36     account.increase_collateral(&position, &token_id, shares);
37     assert!(
38         account.get_assets_num() <= self.internal_config().max_num_assets
39     );
40 }
```

```
41 // 4. updates
42 // update asset
43 self.internal_set_asset(token_id, asset);
44 // update farms
45 self.internal_account_apply_affected_farms(&mut account);
46 // update user account
47 self.internal_set_account(&account_id, account);
48
49 // 5. emit event
50 events::emit::increase_collateral(account_id, amount, token_id, &position);
51 }
```

Listing 2.16: burrowland/contracts/contract/src/actions_only.rs

```
54 pub fn internal_only_repay(
55     &mut self,
56     account_id: &AccountId,
57     token_id: &AccountId,
58     amount: Balance, // inner decimal precision
59 ) {
60     let mut account = self.internal_unwrap_account(&account_id);
61     let mut asset = self.internal_unwrap_asset(token_id);
62
63     // 1. repay
64     let position = REGULAR_POSITION.to_string();
65     let full_repay_shares = account.internal_unwrap_borrowed(&position, token_id);
66     let full_repay_amount = asset.borrowed.shares_to_amount(full_repay_shares, true);
67     let (repay_shares, repay_amount) = if amount >= full_repay_amount {
68         // full repayment
69         (full_repay_shares, full_repay_amount)
70     } else {
71         // partial repayment
72         (asset.borrowed.amount_to_shares(amount, false), amount)
73     };
74     asset.borrowed.withdraw(repay_shares, repay_amount);
75     account.decrease_borrowed(&position, token_id, repay_shares);
76     account.add_affected_farm(FarmId::Borrowed(token_id.clone()));
77     events::emit::repay(&account_id, repay_amount, token_id, &position);
78
79     // 2. remaining supply
80     let remain_amount = amount - repay_amount;
81     if remain_amount > 0 {
82         let mut account_asset = account.internal_get_asset_or_default(token_id);
83         let shares: Shares = asset.supplied.amount_to_shares(remain_amount, false);
84         account_asset.deposit_shares(shares);
85         account.internal_set_asset(&token_id, account_asset);
86         asset.supplied.deposit(shares, remain_amount);
87         events::emit::deposit(&account_id, remain_amount, &token_id);
88     }
89
90     // update asset
91     self.internal_set_asset(token_id, asset);
92     // update farms
```

```
93     self.internal_account_apply_affected_farms(&mut account);
94     // update user account
95     self.internal_set_account(&account_id, account);
96 }
```

Listing 2.17: burrowland/contracts/contract/src/actions_only.rs

Impact A user may be able to withdraw or modify positions while liquidation or forced-close procedures are in progress.

Suggestion Revise the logic accordingly.

2.2.12 Potential underflow in function `internal_storage_balance_of()` after forced storage adjustment

Severity Low

Status Fixed in [Version 8](#)

Introduced by [Version 7](#)

Description The function `internal_force_set_storage()` is intentionally used to reconcile `storage.used_bytes` during asynchronous callbacks, even when the user's prepaid storage balance is temporarily insufficient, and this behavior itself is acceptable. However, after such a forced adjustment, function `internal_storage_balance_of()` computes the available storage balance by subtracting the effective storage cost from `storage.storage_balance`. If the forced `used_bytes` exceeds the prepaid balance, this calculation will underflow, causing the view function `storage_balance_of()` to revert.

Although users can later recover by invoking the function `storage_deposit()`, the deposit must fully cover the storage deficit introduced by the forced adjustment. If the deposited amount is insufficient, the validation in function `assert_storage_covered()` will revert, requiring users to deposit more than the missing amount to successfully restore a valid storage state.

```
90  #[payable]
91  pub fn internal_force_set_storage(&mut self, account_id: &AccountId, mut storage: Storage) {
92      if storage.storage_tracker.bytes_added >= storage.storage_tracker.bytes_released {
93          let extra_bytes_used =
94              storage.storage_tracker.bytes_added - storage.storage_tracker.bytes_released;
95          storage.used_bytes = storage.used_bytes.checked_add(extra_bytes_used).unwrap_or(u64::MAX);
96      } else {
97          let bytes_released =
98              storage.storage_tracker.bytes_released - storage.storage_tracker.bytes_added;
99          storage.used_bytes = storage.used_bytes.checked_sub(bytes_released).unwrap_or(0);
100      };
101      storage.storage_tracker.bytes_released = 0;
102      storage.storage_tracker.bytes_added = 0;
103      self.storage.insert(account_id, &storage.into());
104  }
```

Listing 2.18: burrowland/contracts/contract/src/storage.rs

```

225 fn storage_balance_of(&self, account_id: AccountId) -> Option<StorageBalance> {
226     self.internal_storage_balance_of(&account_id)
227 }

```

Listing 2.19: burrowland/contracts/contract/src/storage.rs

```

105 pub fn internal_storage_balance_of(&self, account_id: &AccountId) -> Option<StorageBalance> {
106     self.internal_get_storage(account_id)
107         .map(|storage| StorageBalance {
108             total: storage.storage_balance.into(),
109             available: U128(
110                 storage.storage_balance
111                     - std::cmp::max(
112                         Balance::from(storage.used_bytes) * env::storage_byte_cost(),
113                         self.storage_balance_bounds().min.0,
114                     ),
115             ),
116         })
117 }

```

Listing 2.20: burrowland/contracts/contract/src/storage.rs

```

47 fn assert_storage_covered(&self) {
48     let storage_balance_needed = Balance::from(self.used_bytes) * env::storage_byte_cost();
49     assert!(
50         storage_balance_needed <= self.storage_balance,
51         "Not enough storage balance"
52     );
53 }

```

Listing 2.21: burrowland/contracts/contract/src/storage.rs

Impact Users may be temporarily unable to query their storage balance via function `storage_balance_of()`. In addition, attempts to recover by topping up storage through function `storage_deposit()` may revert unless the deposited amount fully covers the existing storage deficit, potentially blocking further interaction until sufficient storage fees are provided.

Suggestion Consider guarding the available balance calculation in function `internal_storage_balance_of()` against underflow when `used_bytes` exceeds the prepaid storage balance, or clearly documenting the required recovery behavior and deposit expectations after a forced storage reconciliation.

2.2.13 Incorrect aggregation of intermediate swap outputs

Severity High

Status Fixed in [Version 9](#)

Introduced by [Version 8](#)

Description The function `get_token_out()` computes the expected output amount by summing the `min_amount_out` values of all swap actions whose `token_out` matches the final output token. However, in a multi-hop swap path, intermediate outputs are consumed as inputs of

subsequent swaps, and only the output of the final swap is actually received by the protocol. As a result, intermediate `min_amount_out` values can be repeatedly counted and artificially inflate the returned output amount. Since this inflated value is later used in swap output validation, slippage validation, and position health checks, an attacker can construct a cyclic swap path with repeated intermediate outputs to make an undercollateralized position appear sufficiently collateralized while only delivering a negligible final amount to the protocol.

```

102 pub fn get_token_out(&self) -> (AccountId, Balance) {
103     let RefViTokenReceiverMessage::Execute {
104         referral_id: _,
105         actions,
106         client_echo: _,
107         skip_degen_price_sync: _,
108     } = self;
109     let action = actions.last().unwrap();
110     let token_out = action.get_token_out();
111     let min_amount_out = actions.iter().filter(|a| a.get_token_out() == token_out).map(|a| a.
        get_min_amount_out()).sum();
112     (token_out, min_amount_out)
113 }

```

Listing 2.22: burrowland/contracts/contract/src/margin_trading.rs

Impact An attacker may open positions with artificially inflated position values while the protocol receives insufficient assets from the swap execution, resulting in protocol bad debt and direct loss of reserves.

Suggestion The protocol should strictly validate that the actual value received from swap execution accurately reflects the value consumed from the user position, and ensure that intermediate swap states cannot artificially inflate the perceived output or position value.

2.3 Additional Recommendation

2.3.1 Automatically construct swap indication from the token information

Status Confirmed

Introduced by Version 1

Description In functions `internal_margin_open_position()` and `process_decrease_margin_position()`, a consistency check is required between the token information and the `swap_indication` provided by the user within functions `verify_token_in()` and `verify_token_out()`. However, both of them are specified by the user.

```

174 pub(crate) fn internal_margin_open_position(
175     &mut self,
176     ts: Timestamp,
177     account: &mut MarginAccount,
178     token_c_id: &AccountId,
179     token_c_amount: Balance,
180     token_d_id: &AccountId,
181     token_d_amount: Balance,

```



```
182     token_p_id: &AccountId,
183     min_token_p_amount: Balance,
184     swap_indication: &SwapIndication,
185     prices: &Prices,
186 ) -> EventDataMarginOpen {
187     let pos_id = format!("{}", account.account_id.clone(), ts);
188     assert!(
189         !account.margin_positions.contains_key(&pos_id),
190         "Margin position already exist"
191     );
192
193     let asset_c = self.internal_unwrap_asset(token_c_id);
194     let asset_p = self.internal_unwrap_asset(token_p_id);
195     let mut asset_d = self.internal_unwrap_asset(token_d_id);
196     let margin_config = self.internal_margin_config();
197
198     // check legitimacy: assets legal; swap_indication matches;
199     margin_config.check_pair(&token_d_id, &token_p_id, &token_c_id);
200     let mut swap_detail = self.parse_swap_indication(swap_indication);
201     let ft_d_amount = token_d_amount / 10u128.pow(asset_d.config.extra_decimals as u32);
202     assert!(
203         swap_detail.verify_token_in(token_d_id, ft_d_amount),
204         "token_in check failed"
205     );
206     let ft_p_amount =
207         min_token_p_amount / 10u128.pow(asset_p.config.extra_decimals as u32);
208     assert!(
209         swap_detail.verify_token_out(token_p_id, ft_p_amount),
210         "token_out check failed"
211     );
212
213     // check safty:
214     // min_position_amount reasonable
215     assert!(
216         is_min_amount_out_reasonable(
217             token_d_amount,
218             &asset_d,
219             prices.get_unwrap(&token_d_id),
220             &asset_p,
221             prices.get_unwrap(&token_p_id),
222             min_token_p_amount,
223             margin_config.max_slippage_rate,
224         ),
225         "min_position_amount is too low"
226     );
227     // margin_hf more than 1 + safty_buffer_rate(10%)
228     let mut mt = MarginTradingPosition::new(
229         ts,
230         token_c_id.clone(),
231         asset_c.supplied.amount_to_shares(token_c_amount, false),
232         token_d_id.clone(),
233         token_p_id.clone(),
234     );
```

```
235     mt.token_d_shares = asset_d.margin_debt.amount_to_shares(token_d_amount, true);
236     mt.token_p_amount = min_token_p_amount;
237     assert!(
238         !self.is_mt_liquidatable(&mt, prices, margin_config.min_safty_buffer),
239         "Debt is too much"
240     );
241     assert!(
242         !self.is_mt_forcecloseable(&mt, prices),
243         "Debt is too much"
244     );
245     // leverage rate less than max leverage rate
246     assert!(
247         self.get_mtp_lr(&mt, prices).unwrap()
248             <= BigDecimal::from(margin_config.max_leverage_rate as u32),
249         "Leverage rate is too high"
250     );
251
252     // passes all check, start to open
253     let event = EventDataMarginOpen {
254         account_id: account.account_id.clone(),
255         pos_id: pos_id.clone(),
256         token_c_id: token_c_id.clone(),
257         token_c_amount,
258         token_c_shares: mt.token_c_shares,
259         token_d_id: token_d_id.clone(),
260         token_d_amount,
261         token_p_id: token_p_id.clone(),
262         token_p_amount: min_token_p_amount,
263     };
264     account.withdraw_supply_shares(token_c_id, &mt.token_c_shares);
265     mt.token_d_shares.0 = 0;
266     mt.token_p_amount = 0;
267     asset_d.increase_margin_pending_debt(token_d_amount, margin_config.pending_debt_scale);
268     self.internal_set_asset(token_d_id, asset_d);
269     // TODO: may need to change to store in an unorderedmap in user Account
270     account.margin_positions.insert(pos_id.clone(), mt);
271
272     // step 4: call dex to trade and wait for callback
273     // organize swap action
274     let swap_ref = SwapReference {
275         account_id: account.account_id.clone(),
276         pos_id: pos_id.clone(),
277         amount_in: token_d_amount.into(),
278         op: format!("open"),
279         liquidator_id: None,
280     };
281     swap_detail.set_client_echo(&swap_ref.to_msg_string());
282     let swap_msg = swap_detail.to_msg_string();
283     ext_fungible_token::ext(token_d_id.clone())
284         .with_attached_deposit(1)
285         .with_static_gas(GAS_FOR_FT_TRANSFER_CALL)
286         .ft_transfer_call(
287             swap_indication.dex_id.clone(),
```

```
288         U128(ft_d_amount),
289         None,
290         swap_msg,
291     )
292     .then(
293         Self::ext(env::current_account_id())
294             .with_static_gas(GAS_FOR_FT_TRANSFER_CALL_CALLBACK)
295             .callback_dex_trade(
296                 account.account_id.clone(),
297                 pos_id.clone(),
298                 token_d_amount.into(),
299                 U128(0),
300                 format!("open"),
301             ),
302     );
303     event
304 }
```

Listing 2.23: burrowland/contracts/contract/src/margin_position.rs

```
307 pub(crate) fn process_decrease_margin_position(
308     &mut self,
309     account: &mut MarginAccount,
310     pos_id: &String,
311     token_p_amount: Balance,
312     min_token_d_amount: Balance,
313     swap_indication: &SwapIndication,
314     prices: &Prices,
315     op: String,
316     liquidator_id: Option<AccountId>,
317 ) -> EventDataMarginDecrease {
318     let mut mt = account
319         .margin_positions
320         .get_mut(pos_id)
321         .expect("Position not exist");
322     assert!(
323         !mt.is_locking,
324         "Position is currently waiting for a trading result."
325     );
326     let pre_token_p_amount = mt.token_p_amount;
327     let mut asset_p = self.internal_unwrap_asset(&mt.token_p_id);
328     let asset_d = self.internal_unwrap_asset(&mt.token_d_id);
329     let margin_config = self.internal_margin_config();
330
331     // check swap_indication
332     let mut swap_detail = self.parse_swap_indication(swap_indication);
333     let ft_p_amount =
334         token_p_amount / 10u128.pow(asset_p.config.extra_decimals as u32);
335     assert!(
336         swap_detail.verify_token_in(&mt.token_p_id, ft_p_amount),
337         "token_in check failed"
338     );
339     let ft_d_amount = min_token_d_amount / 10u128.pow(asset_d.config.extra_decimals as u32);
```

```
340     assert!(
341         swap_detail.verify_token_out(&mt.token_d_id, ft_d_amount),
342         "token_out check failed"
343     );
344
345     // min_debt_amount reasonable
346     assert!(
347         is_min_amount_out_reasonable(
348             token_p_amount,
349             &asset_p,
350             prices.get_unwrap(&mt.token_p_id),
351             &asset_d,
352             prices.get_unwrap(&mt.token_d_id),
353             min_token_d_amount,
354             margin_config.max_slippage_rate,
355         ),
356         "min_debt_amount is too low"
357     );
358
359     if op == "close" || op == "liquidate" {
360         // ensure all debt would be repaid
361         // and take holding-position fee into account
362         let total_debt_amount = asset_d
363             .margin_debt
364             .shares_to_amount(mt.token_d_shares, true);
365         let hp_fee = u128_ratio(
366             mt.debt_cap,
367             asset_d.unit_acc_hp_interest - mt.uahpi_at_open,
368             UNIT,
369         );
370         if min_token_d_amount < total_debt_amount + hp_fee {
371             assert_eq!(
372                 mt.token_c_id, mt.token_d_id,
373                 "Can NOT trade under total debt when margin and debt asset are not the same"
374             );
375             let gap_shares = asset_d
376                 .supplied
377                 .amount_to_shares(total_debt_amount + hp_fee - min_token_d_amount, true);
378             assert!(
379                 mt.token_c_shares.0 > gap_shares.0,
380                 "Not all debt could be repaid"
381             );
382         }
383     }
384
385     if op == "liquidate" {
386         assert!(
387             self.is_mt_liquidatable(&mt, prices, margin_config.min_safety_buffer),
388             "Margin position is not liquidatable"
389         );
390     } else if op == "forceclose" {
391         assert!(
392             self.is_mt_forcecloseable(&mt, prices),
```

```
393         "Margin position is not forceclose-able"
394     );
395 }
396
397 // ensure enough position token to trade
398 if token_p_amount > mt.token_p_amount {
399     // try to add some of margin asset into trading
400     assert_eq!(
401         mt.token_c_id, mt.token_p_id,
402         "Not enough position asset balance"
403     );
404     let gap_shares = asset_p
405         .supplied
406         .amount_to_shares(token_p_amount - mt.token_p_amount, true);
407     mt.token_c_shares
408         .0
409         .checked_sub(gap_shares.0)
410         .expect("Not enough position asset balance");
411     asset_p
412         .supplied
413         .withdraw(gap_shares, token_p_amount - mt.token_p_amount);
414     asset_p.margin_position -= mt.token_p_amount;
415     mt.token_p_amount = 0;
416 } else {
417     asset_p.margin_position -= token_p_amount;
418     mt.token_p_amount -= token_p_amount;
419 }
420
421 // prepare to close
422 mt.is_locking = true;
423 self.internal_set_asset(&mt.token_p_id, asset_p);
424 // TODO: mt may be needed to change to store in an unorderedmap in user Account
425
426 let event = EventDataMarginDecrease {
427     account_id: account.account_id.clone(),
428     pos_id: pos_id.clone(),
429     liquidator_id: liquidator_id.clone(),
430     token_p_id: mt.token_p_id.clone(),
431     token_p_amount,
432     token_d_id: mt.token_d_id.clone(),
433     token_d_amount: min_token_d_amount,
434 };
435
436 // step 3: call dex to trade and wait for callback
437 // organize swap action
438 let swap_ref = SwapReference {
439     account_id: account.account_id.clone(),
440     pos_id: pos_id.clone(),
441     amount_in: token_p_amount.into(),
442     op,
443     liquidator_id,
444 };
445 swap_detail.set_client_echo(&swap_ref.to_msg_string());
```

```

446     let swap_msg = swap_detail.to_msg_string();
447     ext_fungible_token::ext(mt.token_p_id.clone())
448         .with_attached_deposit(1)
449         .with_static_gas(GAS_FOR_FT_TRANSFER_CALL)
450         .ft_transfer_call(
451             swap_indication.dex_id.clone(),
452             U128(ft_p_amount),
453             None,
454             swap_msg,
455         )
456         .then(
457             Self::ext(env::current_account_id())
458                 .with_static_gas(GAS_FOR_FT_TRANSFER_CALL_CALLBACK)
459                 .callback_dex_trade(
460                     account.account_id.clone(),
461                     pos_id.clone(),
462                     token_p_amount.into(),
463                     pre_token_p_amount.into(),
464                     format!("decrease"),
465                 ),
466         );
467     event
468 }

```

Listing 2.24: burrowland/contracts/contract/src/margin_position.rs

Suggestion The contract can automatically construct the `swap_indication` based on the token information provided by the user.

Feedback from the Project The code will be optimized as appropriate in subsequent versions.

2.3.2 Use `UnorderedMap` for `margin_positions` instead of `HashMap`

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The current implementation uses a `HashMap` for the `margin_positions` field in the struct `MarginAccount`. This means that every time the `MarginAccount` is deserialized, all the `margin_positions` have to be deserialized as well, resulting in a waste of gas.

```

3  #[derive(BorshSerialize, BorshDeserialize, Serialize, Clone)]
4  #[serde(crate = "near_sdk::serde")]
5  pub struct MarginAccount {
6      /// A copy of an account ID. Saves one storage_read when iterating on accounts.
7      pub account_id: AccountId,
8      /// A list of assets that are supplied by the account (but not used a collateral).
9      /// It's not returned for account pagination.
10     pub supplied: HashMap<TokenId, Shares>,
11     // margin trading related
12     pub margin_positions: HashMap<PosId, MarginTradingPosition>,
13     /// Tracks changes in storage usage by persistent collections in this account.
14     #[borsh_skip]

```

```

15     #[serde(skip)]
16     pub storage_tracker: StorageTracker,
17 }

```

Listing 2.25: burrowland/contracts/contract/src/margin_accounts.rs

Suggestion Use `UnorderedMap` instead of `HashMap` for the `margin_positions` field in the struct `MarginAccount`.

2.3.3 Incorrect error message in `get_token_out()`

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the function `RefV2TokenReceiverMessage::get_token_out()`, "`RefV1TokenReceiverMessage`" is used as an error message, which is incorrect .

```

120 pub fn get_token_out(&self) -> (AccountId, Balance, Option<bool>) {
121     if let RefV2TokenReceiverMessage::Swap {
122         pool_ids: _,
123         output_token,
124         min_output_amount,
125         skip_unwrap_near,
126         client_echo: _,
127     } = self
128     {
129         (
130             output_token.clone(),
131             min_output_amount.0,
132             skip_unwrap_near.clone(),
133         )
134     } else {
135         env::panic_str("Invalid RefV1TokenReceiverMessage");
136     }
137 }

```

Listing 2.26: burrowland/contracts/contract/src/margin_trading.rs

Suggestion Revise "`RefV1TokenReceiverMessage`" to "`RefV2TokenReceiverMessage`".

2.3.4 Lack of check in function `update_asset()`

Status Fixed in [Version 5](#)

Introduced by [Version 4](#)

Description The protocol `owner` can update the global config of a given token via the function `update_asset()`. When the `extra_decimals` parameter is modified, the protocol does not account for the fees already accumulated for `beneficiaries`. During `withdraw_beneficiary_fee()`, the protocol adjusts recorded amounts based on the current `extra_decimals` to calculate the actual transferable value. If `extra_decimals` changes, this may result in inconsistencies between recorded fees and the actual withdrawal amount.

```
213 pub fn update_asset(&mut self, token_id: AccountId, asset_config: AssetConfig) {
214     assert_one_yocto();
215     asset_config.assert_valid();
216     self.assert_owner();
217     let mut asset = self.internal_unwrap_asset(&token_id);
218     if asset.config.extra_decimals != asset_config.extra_decimals {
219         assert!(
220             asset.borrowed.balance == 0 && asset.supplied.balance == 0 &&
221             asset.margin_debt.balance == 0 && asset.margin_pending_debt == 0 && asset.
222                 margin_position == 0 &&
223             asset.prot_fee == 0 && asset.reserved == 0,
224             "Can't change extra decimals if any of the balances are not 0"
225         );
226     }
227     asset.config = asset_config;
228     self.internal_set_asset(&token_id, asset);
229 }
```

Listing 2.27: burrowland/contracts/contract/src/config.rs

Suggestion Revise the logic accordingly.

2.3.5 Remove redundant code in function

`internal_decrease_collateral_for_gap()`

Status Fixed in [Version 6](#)

Introduced by [Version 5](#)

Description A user can invoke the function `simple_withdraw()` to withdraw assets they hold in the protocol. If the user's internal balance is insufficient to cover the requested withdrawal amount, the logic proceeds to function `internal_decrease_collateral_for_gap()`, which attempts to release part of the user's collateral while ensuring the position remains healthy.

Within this function, the parameter `involved_tokens` represents the number of tokens involved in the user's position. When `involved_tokens` equals 0, indicating that the user has no open position, execution enters the `else` branch. This branch ultimately fails since no valid position can be found. Therefore, the existence of this branch introduces unnecessary control flow complexity.

```
834 pub fn simple_withdraw(&mut self, token_id: AccountId, amount_with_inner_decimal: U128,
835     recipient_id: Option<AccountId>) -> Promise {
836     assert_one_yocto();
837     let account_id = env::predecessor_account_id();
838     let recipient_id = recipient_id.unwrap_or(account_id.clone());
839     let mut account = self.internal_unwrap_account(&account_id);
840
841     assert!(!token_id.to_string().starts_with(SHADOW_V1_TOKEN_PREFIX));
842
843     // check if has enough token in user's supply
844     let mut gap = amount_with_inner_decimal.0;
```



```
845     if let Some(shares) = account.supplied.get(&token_id) {
846         let asset = self.internal_unwrap_asset(&token_id);
847         let amount = asset.supplied.shares_to_amount(shares.clone(), false);
848         gap = gap.saturating_sub(amount);
849     }
850
851     if gap > 0 {
852         // try use collateral asset
853         if let Some(pyth_promise) = self.internal_decrease_collateral_for_gap(&account_id, &mut
            account, &token_id, amount_with_inner_decimal.0, gap, &recipient_id) {
854             return pyth_promise;
855         }
856     }
857
858     // finally withdraw from supply
859     let promise = self.internal_simple_withdraw(&account_id, &mut account, &token_id,
        amount_with_inner_decimal.0, &recipient_id);
860     self.internal_set_account(&account_id, account);
861     promise
862 }
```

Listing 2.28: burrowland/contracts/contract/src/action.rs

```
936 pub fn internal_decrease_collateral_for_gap(
937     &mut self,
938     account_id: &AccountId,
939     account: &mut Account,
940     token_id: &AccountId,
941     total_amount: u128,
942     gap_amount: u128,
943     recipient_id: &AccountId,
944 ) -> Option<Promise> {
945     let actions = vec![Action::DecreaseCollateral(AssetAmount {
946         token_id: token_id.clone(),
947         amount: Some(gap_amount.into()),
948         max_amount: None
949     })];
950     let involved_tokens: Vec<AccountId> = self.involved_tokens(&account, &actions);
951     if involved_tokens.len() > 0 {
952         assert!(self.internal_config().enable_pyth_oracle, "Pyth oracle disabled");
953         let (promise_token_ids, default_prices) = self.prepare_promise_tokens(&involved_tokens)
            ;
954         if promise_token_ids.len() > 0 {
955             let (all_promise_flags, promise) = self.generate_flags_and_promise(&
                promise_token_ids);
956             let callback_gas = env::prepaid_gas() - (GAS_FOR_GET_PRICE) * all_promise_flags.len
                () as u64 - GAS_FOR_BUFFER;
957             Some(promise.then(
958                 Self::ext(env::current_account_id())
959                     .with_static_gas(callback_gas)
960                     .callback_simple_withdraw_with_pyth(
961                         account_id.clone(),
962                         involved_tokens,
```

```
963             all_promise_flags,
964             token_id.clone(),
965             total_amount.into(),
966             gap_amount.into(),
967             recipient_id.clone(),
968             default_prices)
969         ))
970     } else {
971         self.internal_simple_decrease_collateral(
972             account_id,
973             account,
974             token_id,
975             gap_amount,
976             Prices::from_prices(default_prices));
977         None
978     }
979 } else {
980     self.internal_simple_decrease_collateral(
981         account_id,
982         account,
983         token_id,
984         gap_amount,
985         Prices::new());
986     None
987 }
988 }
```

Listing 2.29: burrowland/contracts/contract/src/action.rs

Suggestion Simplify the logic by directly reverting when `involved_tokens == 0`.

2.4 Notes

2.4.1 Potential centralization risks

Introduced by [Version 1](#)

Description In this project, several privileged roles (e.g., `owner`) can conduct sensitive operations, which introduces potential centralization risks. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

2.4.2 Collateral tracking approximation due to legacy asset accounting

Introduced by [Version 6](#)

Description Earlier protocol versions did not separately track how much of each supplied asset was designated as collateral. When a user deposited assets, the amount was directly added to the asset's `supplied.balance`. If the user later used part of that supply as collateral, the protocol simply increased the collateral value without recording the precise portion already used.

As a result, when enforcing `supply_limit`, the protocol assumes that all supplied assets could potentially be used as collateral. Therefore, the collateral cap check compares `supply_limit.balance` against `supply_limit` in functions `internal_increase_collateral()` and `internal_margin_increase_collateral()`.

Although this approximation may not perfectly reflect the actual collateralized amount, it ensures protocol safety and operational continuity. The `admin` can still fine-tune each asset's `supply_limit` through privileged actions to maintain balance and prevent over-collateralization.

